

Experiment No. 4

Experiment Name - Frequency Domain Image Filtering using Fourier Transform

Steps

NAME : Atul Krishna

COURSE : B.Tech CSE

SECTION - B

SUBJECT : Computer Vision

ROLL NO. : 14

CU ID : CU24250231

1. Import the required libraries and load a grayscale image for processing.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Uploading image
uploaded = files.upload()

# Get the uploaded image filename
filename = list(uploaded.keys())[0]

# Read the color image
image = cv2.imread(filename)

# Check if image was loaded successfully
if image is None:
    print("Error: Image could not be loaded.")
else:
    # Convert color image to grayscale
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    print("Image loaded successfully!")
    print("Original Image Shape:", image.shape)
    print("Grayscale Image Shape:", gray_image.shape)

    # Display original and grayscale images
    plt.figure(figsize=(4, 3))

    plt.subplot(1, 2, 1)
    plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
    plt.title("Original Color Image")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(gray_image, cmap="gray")
    plt.title("Grayscale Image")
    plt.axis("off")

    plt.tight_layout()
    plt.show()
```

my photo.jpg

my photo.jpg(image/jpeg) - 12084 bytes, last modified: 8/20/2026 - 100% done

Saving my photo.jpg to my photo (2).jpg

Image loaded successfully!

Original Image Shape: (300, 217, 3)

Grayscale Image Shape: (300, 217)

Original Color Image

Grayscale Image



2. Compute the Discrete Fourier Transform (DFT) of the image using OpenCV or NumPy.

```
# Converting grayscale image to floating-point format
gray_float = np.float32(gray_image)

# Computing the Discrete Fourier Transform (DFT) using OpenCV
dft = cv2.dft(gray_float, flags=cv2.DFT_COMPLEX_OUTPUT)

# Displaying the size of the DFT
print("DFT computed successfully!")
print("DFT Shape:", dft.shape)

DFT computed successfully!
DFT Shape: (300, 217, 2)
```

3. Shift the zero-frequency component to the center of the frequency spectrum for visualization.

```
# Shift the zero-frequency component to the center
dft_shift = np.fft.fftshift(dft)

print("Zero-frequency component shifted to the center.")
print("Shifted DFT Shape:", dft_shift.shape)

Zero-frequency component shifted to the center.
Shifted DFT Shape: (300, 217, 2)
```

4. Display and analyze the magnitude spectrum of the transformed image.

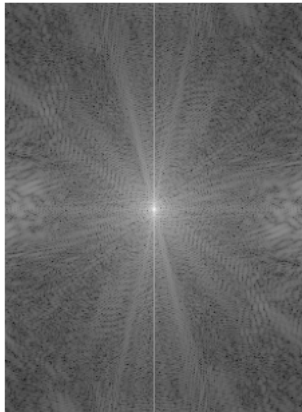
```
# Calculating the magnitude of the frequency spectrum
magnitude_spectrum = cv2.magnitude(
    dft_shift[:, :, 0],
    dft_shift[:, :, 1]
)

# Applying logarithmic transformation for better visualization
magnitude_spectrum = 20 * np.log(magnitude_spectrum + 1)

# Displaying the magnitude spectrum
plt.figure(figsize=(5, 4))

plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Magnitude Spectrum of the Image")
plt.xlabel("Frequency - u")
plt.ylabel("Frequency - v")
plt.axis("off")
plt.show()
```

Magnitude Spectrum of the Image



5. Design and apply a Low-Pass Frequency Filter to suppress high-frequency components and reduce image noise.

```
# Get image dimensions
rows, cols = gray_image.shape

# Find the center of the frequency spectrum
crow = rows // 2
ccol = cols // 2

# Set the radius of the Low-Pass Filter
radius = 50

# Create a mask with two channels
low_pass_mask = np.zeros((rows, cols, 2), np.float32)

# Create a circular Low-Pass Filter
# The center region (low frequencies) is kept as 1
```

```

cv2.circle(
    low_pass_mask,
    (ccol, crow),
    radius,
    (1, 1),
    -1
)

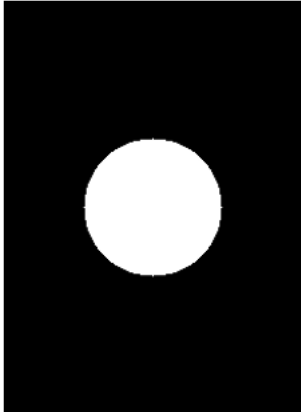
# Apply the Low-Pass Filter to the shifted DFT
low_pass_dft = dft_shift * low_pass_mask

# Display the Low-Pass Filter mask
plt.figure(figsize=(4, 4))
plt.imshow(low_pass_mask[:, :, 0], cmap="gray")
plt.title("Low-Pass Frequency Filter Mask")
plt.axis("off")
plt.show()

print("Low-Pass Filter applied successfully!")
print("Filter Radius:", radius)

```

Low-Pass Frequency Filter Mask



Low-Pass Filter applied successfully!
Filter Radius: 50

6. Design and apply a High-Pass Frequency Filter to suppress low-frequency components and enhance edges and fine details.

```

# Create a High-Pass Filter mask
# Initially, all frequency components are set to 1
high_pass_mask = np.ones((rows, cols, 2), np.float32)

# Remove the low-frequency components from the center
# The center circular region is set to 0
cv2.circle(
    high_pass_mask,
    (ccol, crow),
    radius,
    (0, 0),
    -1
)

# Applying the High-Pass Filter to the shifted DFT
high_pass_dft = dft_shift * high_pass_mask

# Displaying the High-Pass Filter mask
plt.figure(figsize=(4, 4))

plt.imshow(high_pass_mask[:, :, 0], cmap="gray")
plt.title("High-Pass Frequency Filter Mask")
plt.axis("off")
plt.show()

print("High-Pass Filter applied successfully!")
print("Filter Radius:", radius)

```

High-Pass Frequency Filter Mask



High-Pass Filter applied successfully!
Filter Radius: 50

7. Perform the Inverse Fourier Transform (IDFT) to reconstruct the filtered images in the spatial domain.

```
# Low-Pass Filtered Image
# Shift the low-pass frequency components back
low_pass_inverse_shift = np.fft.ifftshift(low_pass_dft)

# Perform Inverse DFT
low_pass_image = cv2.idft(low_pass_inverse_shift)

# Calculate magnitude of the reconstructed image
low_pass_image = cv2.magnitude(
    low_pass_image[:, :, 0],
    low_pass_image[:, :, 1]
)

# Normalize the image to 0-255
low_pass_image = cv2.normalize(
    low_pass_image,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

# Convert to 8-bit image
low_pass_image = np.uint8(low_pass_image)

# High-Pass Filtered Image
# Shift the high-pass frequency components back
high_pass_inverse_shift = np.fft.ifftshift(high_pass_dft)

# Perform Inverse DFT
high_pass_image = cv2.idft(high_pass_inverse_shift)

# Calculate magnitude of the reconstructed image
high_pass_image = cv2.magnitude(
    high_pass_image[:, :, 0],
    high_pass_image[:, :, 1]
)

# Normalize the image to 0-255
high_pass_image = cv2.normalize(
    high_pass_image,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

# Convert to 8-bit image
high_pass_image = np.uint8(high_pass_image)

# Display the reconstructed images
plt.figure(figsize=(5, 4))

plt.subplot(1, 2, 1)
plt.imshow(low_pass_image, cmap="gray")
plt.title("Low-Pass Filtered Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(high_pass_image, cmap="gray")
plt.title("High-Pass Filtered Image")
plt.axis("off")
```

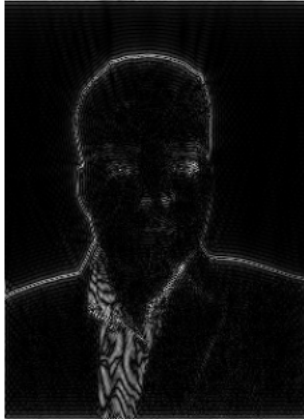
```
plt.tight_layout()
plt.show()

print("Inverse Fourier Transform completed successfully!")
print("Low-Pass and High-Pass images reconstructed.")
```

Low-Pass Filtered Image



High-Pass Filtered Image



Inverse Fourier Transform completed successfully!
Low-Pass and High-Pass images reconstructed.

8. Compare the original image, frequency spectrum, low-pass filtered image, and highpass filtered image.

```
# Compare Original, Frequency Spectrum, Low-Pass and High-Pass Filtered Images

plt.figure(figsize=(6, 4))

# 1. Original Grayscale Image
plt.subplot(2, 2, 1)
plt.imshow(gray_image, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")

# 2. Magnitude Spectrum
plt.subplot(2, 2, 2)
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Magnitude Spectrum")
plt.axis("off")

# 3. Low-Pass Filtered Image
plt.subplot(2, 2, 3)
plt.imshow(low_pass_image, cmap="gray")
plt.title("Low-Pass Filtered Image")
plt.axis("off")

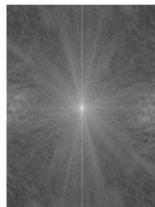
# 4. High-Pass Filtered Image
plt.subplot(2, 2, 4)
plt.imshow(high_pass_image, cmap="gray")
plt.title("High-Pass Filtered Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original Grayscale Image



Magnitude Spectrum



Low-Pass Filtered Image



High-Pass Filtered Image



9. Analyze the impact of frequency domain filtering on image smoothness, edge enhancement, and noise removal.

```
# Display the three images for analysis
plt.figure(figsize=(5, 4))

# Original Image
plt.subplot(1, 3, 1)
plt.imshow(gray_image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

# Low-Pass Filtered Image
plt.subplot(1, 3, 2)
plt.imshow(low_pass_image, cmap="gray")
plt.title("Low-Pass Filtered")
plt.axis("off")

# High-Pass Filtered Image
plt.subplot(1, 3, 3)
plt.imshow(high_pass_image, cmap="gray")
plt.title("High-Pass Filtered")
plt.axis("off")
plt.tight_layout()
plt.show()
```

Original Image Low-Pass Filtered High-Pass Filtered



Print analysis

Frequency domain filtering has a significant effect on image smoothness, noise removal, edge enhancement, and fine-detail preservation.

1. Image Smoothness

The Low-Pass Filter suppresses high-frequency components of the image. As a result, the output image becomes smoother and some fine details are reduced.

2. Noise Removal

High-frequency components may contain noise and unwanted intensity variations. The Low-Pass Filter suppresses these components and therefore helps in reducing high-frequency noise.

3. Edge Enhancement

Edges and fine details contain high-frequency components. The High-Pass Filter suppresses low-frequency components and preserves high-frequency components, making edges and fine details more prominent.

4. Overall Effect Low-Pass Filtering: Produces a smoother image and helps in noise reduction. High-Pass Filtering: Enhances edges and fine details.

Therefore, frequency domain filtering allows selective manipulation of different frequency components of an image and is useful for image smoothing, noise reduction, edge detection, and feature enhancement.

10. Document the observations and discuss the advantages of frequency domain filtering in practical computer vision applications.

Observations

1. The color image was first converted into a grayscale image for frequency domain processing.
2. The Discrete Fourier Transform (DFT) converted the image from the spatial domain into the frequency domain.
3. The magnitude spectrum showed the distribution of frequency components. Low-frequency components were concentrated near the center, while high-frequency components appeared farther from the center.
4. The Low-Pass Filter suppressed high-frequency components and preserved low-frequency components. The resulting image appeared smoother, with reduced fine details and high-frequency noise.
5. The High-Pass Filter suppressed low-frequency components and preserved high-frequency components. As a result, edges and fine details became more prominent.
6. The Inverse Fourier Transform (IDFT) successfully reconstructed the filtered images back into the spatial domain.

Advantages of Frequency Domain Filtering

- **Effective frequency selection:** It allows specific frequency components to be preserved or suppressed according to the application.
- **Noise reduction:** Low-Pass filtering can reduce high-frequency noise and unwanted fine variations.
- **Edge enhancement:** High-Pass filtering can emphasize edges and fine image details.
- **Image enhancement:** Frequency components can be manipulated to improve the visual quality and important features of an image.
- **Useful for image restoration:** Frequency domain techniques can be used to reduce certain types of noise and degradation.
- **Wide range of applications:** Frequency domain filtering is useful in medical imaging, satellite image processing, biometric systems, image restoration, and computer vision.

Practical Computer Vision Applications

Frequency domain filtering is used in practical computer vision applications such as:

- Medical image enhancement and analysis
- Satellite and remote sensing image processing
- Face and biometric recognition
- Image restoration and noise reduction
- Edge and feature enhancement
- Industrial image inspection
- Object detection and image preprocessing

Conclusion

The experiment successfully demonstrated image filtering in the frequency domain using the Fourier Transform. The Low-Pass Filter was useful for smoothing and noise reduction, while the High-Pass Filter enhanced edges and fine details. Frequency domain filtering provides an effective way to analyze and selectively manipulate image frequency components and is therefore an important technique in digital image processing and computer vision.

Questions and Answers

1. What is the Fourier Transform? Why is it important in digital image processing?

Answer - The **Fourier Transform (FT)** is a mathematical technique that converts an image from the **spatial domain** into the **frequency domain**. It represents an image as a combination of different frequency components.

It is important in digital image processing because it allows us to analyze and manipulate different frequency components separately. It is widely used for **image filtering, noise reduction, image enhancement, image restoration, and edge/feature analysis.**

2. Differentiate between the spatial domain and the frequency domain.

Answer -

Spatial Domain	Frequency Domain
Image is represented using pixel intensity values.	Image is represented using frequency components.
Operations are performed directly on pixels.	Operations are performed on frequency components.
Examples include blurring, sharpening, and thresholding.	Examples include Low-Pass and High-Pass filtering.
Local pixel information is directly processed.	Information about variations and frequencies is analyzed.
Usually easier to implement for simple operations.	Useful for analyzing and selectively modifying frequency components.

3. What is the significance of the Discrete Fourier Transform (DFT) in image processing?

Answer - The **Discrete Fourier Transform (DFT)** converts a digital image from the spatial domain into the frequency domain. It provides information about the frequency components present in an image.

The DFT is significant because it helps in:

- Image filtering
- Noise reduction
- Image enhancement
- Image restoration
- Edge and feature analysis
- Frequency spectrum visualization

In practical image processing, the DFT allows specific frequency components to be suppressed or preserved.

4. Explain the purpose of shifting the zero-frequency component to the center of the frequency spectrum.

Answer - After applying the Fourier Transform, the **zero-frequency component** is normally located at the corners of the frequency spectrum.

The function `fftshift()` is used to move the zero-frequency component to the **center** of the spectrum.

This makes the frequency spectrum easier to visualize and analyze. After shifting, the **low-frequency components appear near the center**, while the high-frequency components appear farther away from the center.

5. Compare Low-Pass Frequency Filters and High-Pass Frequency Filters with suitable applications.

Answer -

Low-Pass Filter (LPF)	High-Pass Filter (HPF)
Preserves low-frequency components.	Preserves high-frequency components.
Suppresses high-frequency components.	Suppresses low-frequency components.
Produces a smoother image.	Enhances edges and fine details.
Helps reduce high-frequency noise.	Helps emphasize edges and rapid intensity changes.
Used for image smoothing and noise reduction.	Used for edge enhancement and feature extraction.

Applications:

- **Low-Pass Filter:** Image smoothing, noise reduction, and removal of small unwanted variations.
- **High-Pass Filter:** Edge enhancement, detail enhancement, and feature extraction.

6. What is the role of the Inverse Fourier Transform (IDFT) in image reconstruction?

Answer - The **Inverse Discrete Fourier Transform (IDFT)** converts an image from the frequency domain back into the **spatial domain**. After applying a frequency-domain filter, the filtered frequency components are converted back into an image using IDFT. Therefore, IDFT is essential for reconstructing and displaying the filtered image in its normal spatial representation.

7. Why is frequency domain filtering preferred for certain image enhancement tasks?

Answer - Frequency domain filtering is preferred for certain tasks because it provides direct control over the different frequency components of an image.

It allows us to:

- Remove unwanted frequency components.
- Reduce certain types of noise.
- Enhance edges and fine details.
- Smooth images by suppressing high frequencies.
- Analyze the frequency characteristics of an image.

It is particularly useful when the enhancement operation is naturally expressed in terms of frequency components.

8. Mention four real-world applications where Fourier Transform is used in computer vision and image analysis.

Four real-world applications are:

1. **Medical Imaging** – Enhancement and restoration of medical images such as MRI and CT images.
2. **Satellite Image Processing** – Noise reduction and enhancement of satellite and remote-sensing images.
3. **Biometric Systems** – Feature enhancement in fingerprint, face, and iris images.
4. **Industrial Inspection** – Detection and analysis of patterns, defects, and surface irregularities in manufactured products.

Other applications include image restoration, microscopy, astronomy, and computer vision preprocessing.

9. Compare frequency domain filtering with spatial domain filtering based on computational efficiency and practical applications.

Answer -

Frequency Domain Filtering	Spatial Domain Filtering
Works on frequency components after Fourier transformation.	Works directly on image pixels.
Requires forward and inverse Fourier transforms.	Usually does not require transformations.
Efficient for large convolution-based filters using FFT-based methods.	Simple and efficient for small kernels.
Provides direct control over frequency components.	Provides direct control over local pixel neighborhoods.
Useful for periodic noise removal, image restoration, and frequency-based enhancement.	Useful for smoothing, sharpening, edge detection, and basic image enhancement.

For small filters, spatial-domain filtering can be simpler and faster. For large filters or operations involving specific frequency components, frequency-domain filtering can be computationally advantageous, especially when FFT-based methods are used.

10. How does frequency domain filtering improve the performance of image restoration and feature extraction techniques?

Answer - Frequency domain filtering can improve image restoration by selectively suppressing unwanted frequency components caused by **noise or image degradation** while preserving useful information.

For feature extraction, High-Pass filtering can emphasize **edges, fine details, and rapid intensity changes**, making important features easier to detect.

Therefore, frequency domain filtering can improve the quality of the input image before further computer vision tasks such as **edge detection, object detection, image segmentation, and feature extraction**.

In image restoration, it helps separate useful image information from certain unwanted frequency components, while in feature extraction it can make important structural details more prominent.

Overall Conclusion

The Fourier Transform provides an effective way to analyze and manipulate images in the frequency domain. **Low-Pass Filters** are mainly useful for smoothing and noise reduction, while **High-Pass Filters** are useful for edge and detail enhancement. Together with DFT and IDFT, frequency domain techniques form an important part of digital image processing and computer vision.